Predicting Car Accident Likelihood and Estimating Crash Costs for an Auto Insurance Company

Teddy Kelly

Table of contents

Motivation

Every day, millions of Americans from all different backgrounds get in their cars to drive to work, school, shopping, etc. Unfortunately, while the vast majority of drivers get to their destinations without any problems, not a day goes by without the occurrence of car accidents. Some are minor while others are fatal, however, all car accidents result in some type of damage requiring costs to be paid. The goal of this project is to develop a binary logistic regression model that can accurately predict the likelihood of someone getting in a car accident based on their background, and a multiple linear regression model that will predict the expected cost of the accident if the person does get in an accident.

Setup

Clear the environment

```
train_data <- read.csv(
  "/Users/teddykelly/Downloads/insurance-training-data2-2.csv")

eval_data <- read.csv(
  "/Users/teddykelly/Downloads/insurance-testing-data2-2.csv")
```

Changing the variable names both datasets to be more readable.

```
#Converting variables with multiple categories into ordered factors before
# converting them to numerics
train_data$CAR_TYPE <- factor(
  train_data$CAR_TYPE,
  levels = c("Minivan", "Panel Truck", "Pickup", "Sports Car", "Van",
             "z_SUV"),
  ordered = TRUE
)

train_data$EDUCATION <- factor(
  train_data$EDUCATION,
  levels = c("<High School", "Bachelors", "Masters", "PhD",
             "z_High School"),
  ordered = TRUE
)

train_data$JOB <- factor(
```

```
  train_data$JOB,
  levels = c("", "Clerical", "Doctor", "Home Maker", "Lawyer", "Manager",
             "Professional", "Student", "z_Blue Collar"),
  ordered = TRUE
)

train_data <- train_data |> transmute(
  crash_flag         = TARGET_FLAG,
  crash_cost         = TARGET_AMT,
  driver_age         = AGE,
  car_value          = as.numeric(gsub("[$,]", "", BLUEBOOK)),
  car_age            = as.numeric(CAR_AGE),
  car_type           = CAR_TYPE,
  car_use            = ifelse(CAR_USE=='Private', 1, 0),
  past_claims_count  = CLM_FREQ,
  educ_level         = EDUCATION,
  num_children_home  = HOMEKIDS,
  home_value         = as.numeric(gsub("[$,]", "", HOME_VAL)),
  income             = as.numeric(gsub("[$,]", "", INCOME)),
  job_category       = JOB,
  teen_drivers       = KIDSDRIV,
  married            = ifelse(MSTATUS=='Yes', 1, 0),
  mvr_points         = MVR_PTS,
  past_claims_amount = as.numeric(gsub("[$,]","", OLDCLAIM)),
  single_parent      = ifelse(PARENT1=='Yes', 1, 0),
  red_car            = ifelse(RED_CAR=='yes', 1, 0),
  license_revoked    = ifelse(train_data$REVOKED=='Yes', 1, 0),
  gender             = ifelse(train_data$SEX=='M', 1, 0),
  policy_tenure      = TIF,
  commute_time       = TRAVTIME,
  urban_rural        = ifelse(URBANICITY=='Highly Urban/ Urban', 1, 0),
  years_on_job       = YOJ
)
```

1 Data Exploration

Section Goal

Before developing any sophisticated models for predicting the likelihood of a crash and how much the crashes, we must first understand the structure of the training dataset. This involves understanding the following:

1. The size of the data
2. Whether there are missing values
3. Distribution of data for each variable

Training Dataset Structure

- The total data contains about 8,000 observations and 25 variables of interest measured at a fixed point in time (cross-sectional data)
- Each observation represents a customer at an auto insurance company. Each variable represents information about the corresponding customer. Note that since there are so many variables, we must make decisions about which variables to include in the regressions.
- The data is split into two groups:
  - Training dataset: Used to train or fit the models on. It contains 6,528 observations (~80% of the data)
  - Evaluation dataset: Used to evaluate the performance of the trained models on new unseen data. It contains 1,633 observations (~20% of the data)
- Below is a table displaying each of the 25 variables with their names as how they appear in the training data and a description of what they mean.

Table 1: Variable Names and Definitions

Variable Name	**Variable Meaning**
crash_flag	Indicates whether the car was involved in a crash (1 = Yes, 0 = No)
crash_cost	Cost of the accident if person was in a crash
driver_age	Age of the driver
car_value	Estimated value of the vehicle (Blue Book value)
car_age	Age of the vehicle in years
car_type	Type of vehicle (e.g., SUV, Sedan, Sports)
car_use	How the vehicle is used (e.g., Commercial or Private)
past_claims_count	Number of past claims filed in the last 5 years
educ_level	Driver's highest education level

Variable Name	Variable Meaning
num_children_home	Number of children living at home
home_value	Estimated value of the driver's home
income	Driver's annual income
job_category	Job category or occupation type
teen_drivers	Number of teenage drivers using the car
married	Marital status of the driver
mvr_points	Motor Vehicle Record (MVR) points — measure of violations
past_claims_amount	Total claim amount from the past 5 years
single_parent	Indicates if the driver is a single parent
red_car	Indicates if the vehicle is red
license_revoked	Indicates if the driver's license was revoked in the past 7 years
gender	Gender of the driver
policy_tenure	Number of years the policy has been active
commute_time	Average commute time or distance to work
urban_rural	Urban or rural location indicator
years_on_job	Number of years the driver has been at their current job

1.2 Summary Statistics

The summary statistics on the numerical and binary variables are seen below. As we can see, there are no longer any missing variables which will allow us to build our models to make predictions. Notice that I did not include any of the categorical variables like `car_type`, `educ_level`, and `job_category`. I will display the summary statistics for these variables separately to make more sense of them.

```
labels <- c(
  "Crash Indicator",
  "Crash Cost",
  "Driver Age",
  "Vehicle Value",
  "Vehicle Age in years",
  "Vehicle Use",
  "Past Claims count",
  "Children at Home",
  "Home Value",
  "Annual Income",
  "Teen Drivers count",
  "Marital Status",
  "MVR Points",
  "Past Claims",
```

```
  "Single Parent",
  "Red Car Indicator",
  "License Revoked",
  "Driver Gender",
  "Policy Tenure in years",
  "Commute Time/Distance",
  "Urbanicity",
  "Years on Job"
)

stargazer(train_data, type = "text",
          digits = 2, median = T, covariate.labels = labels)
```

```
================================================================================
Statistic                  N      Mean      St. Dev.    Min  Median      Max
--------------------------------------------------------------------------------
Crash Indicator        6,528     0.26        0.44      0      0         1
Crash Cost             6,528  1,466.62   4,545.65   0.00   0.00   107,586.10
Driver Age             6,525   44.85        8.65     16     45         81
Vehicle Value          6,528 15,641.76   8,381.49  1,500 14,370     69,740
Vehicle Age in years   6,129    8.32        5.72     -3      8         27
Vehicle Use            6,528    0.63        0.48      0      1         1
Past Claims count      6,528    0.80        1.16      0      0         5
Children at Home       6,528    0.72        1.11      0      0         5
Home Value             6,160 154,298.40 128,874.60   0   160,680  885,282
Annual Income          6,174 61,649.32  47,658.41    0   53,276   367,030
Teen Drivers count     6,528    0.17        0.51      0      0         4
Marital Status         6,528    0.60        0.49      0      1         1
MVR Points             6,528    1.70        2.14      0      1         13
Past Claims            6,528  4,119.32   8,924.67    0      0      57,037
Single Parent          6,528    0.13        0.34      0      0         1
Red Car Indicator      6,528    0.29        0.45      0      0         1
License Revoked        6,528    0.13        0.33      0      0         1
Driver Gender          6,528    0.46        0.50      0      0         1
Policy Tenure in years 6,528    5.37        4.15      1      4         25
Commute Time/Distance  6,528   33.44       15.97      5     33        142
Urbanicity             6,528    0.79        0.40      0      1         1
Years on Job           6,153   10.51        4.10      0     11        23
--------------------------------------------------------------------------------
```

Summary Statistics Analysis:

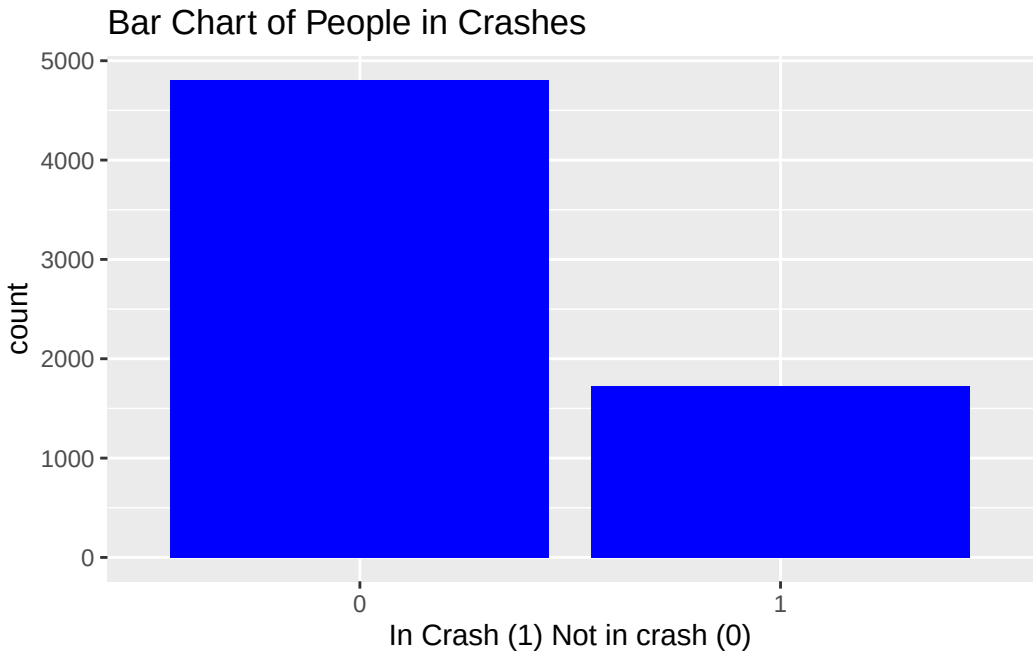
- Dependent Variables
  - `crash_flag` is a binary variable that determines whether a person was involved in an accident. 1 indicates that the observation was in a crash and 0 means they were not. The mean value is 0.26 which means that about 26% of the people in the survey did not crash their car.
  - `crash_cost` has a mean value of $1,466.62, meaning that the average cost of accidents for both people involved and not involved in car crashes is about $1,466. The cost of car crashes is heavily skewed to the right as the median value of `crash_cost` is zero which confirms what we found above that most of the people in this sample were not involved in car crashes. I will have to filter the data to only include observations who were in a crash when running the multiple linear regression.
- Key Independent Variables
  - `driver_age` is very symmetrical as the mean and median ages of the driver are both around 45 years old. This will be an intriguing variable to look at since it's usually believed that young drivers are more reckless and also that the elderly are not great drivers.
  - `income` will be interesting to investigate because we would assume that wealthier people will be better drivers. However, wealthy people could own fancy sports cars which could actually be more dangerous to operate. Income is of course right-skewed which is expected.
- Missing Values
  - `driver_age`, `car_age`, `home_value`, `income`, and `years_on_job` all have missing observations.
  - I will address these missing values in the data preparation section before building the logistic and linear regression models.

1.3 Histograms and Bar Charts

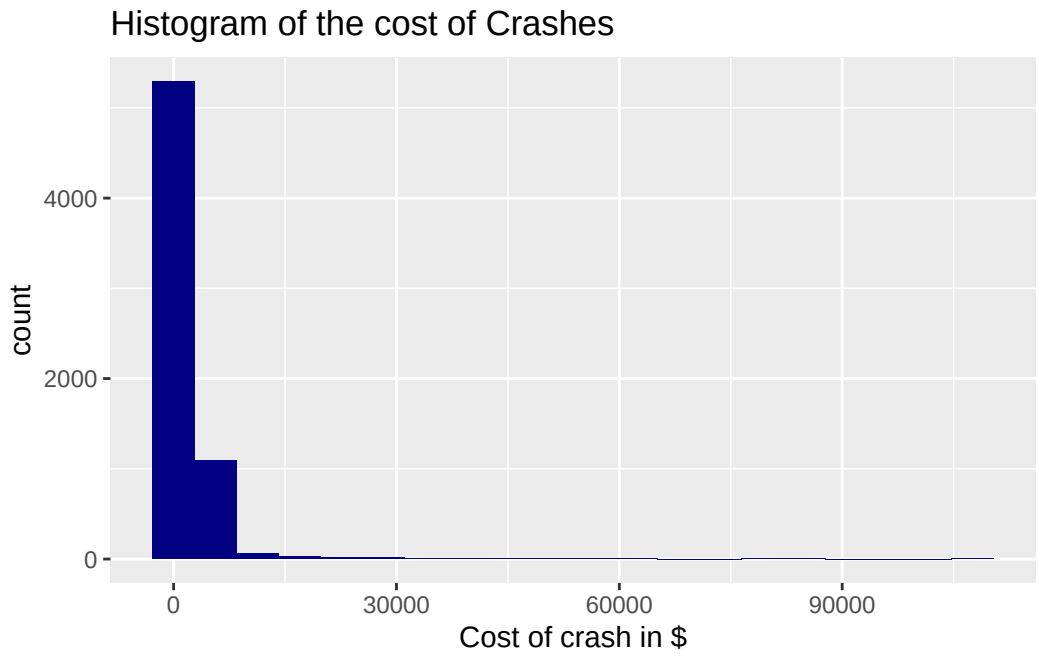
In the previous section, we looked at the summary statistics of the numerical and dummy variables in the dataset. Now, we will visually identify the distributions of each of the numerical and binary variables using histograms.

Dependent Variables:

```r
# Crash Indicator Bar Chart

ggplot(data = train_data,
       mapping = aes(x = factor(crash_flag))) +
  labs(title = "Bar Chart of People in Crashes",
       x = "In Crash (1) Not in crash (0)") +
  geom_bar(fill = "blue")
```




```r
ggplot(data = train_data,
       mapping = aes(x = crash_cost))+
 labs(title = "Histogram of the cost of Crashes", x = "Cost of crash in $")+
 geom_histogram(bins = 20, fill = "navy")
```



- Nearly 5,000 customers did not get into an accident, while under 2,000 did get into a car crash.
- The distribution of the cost of the crashes is heavily right skewed. There are potentially some outliers that I will deal with either through imputation or a log transformation.

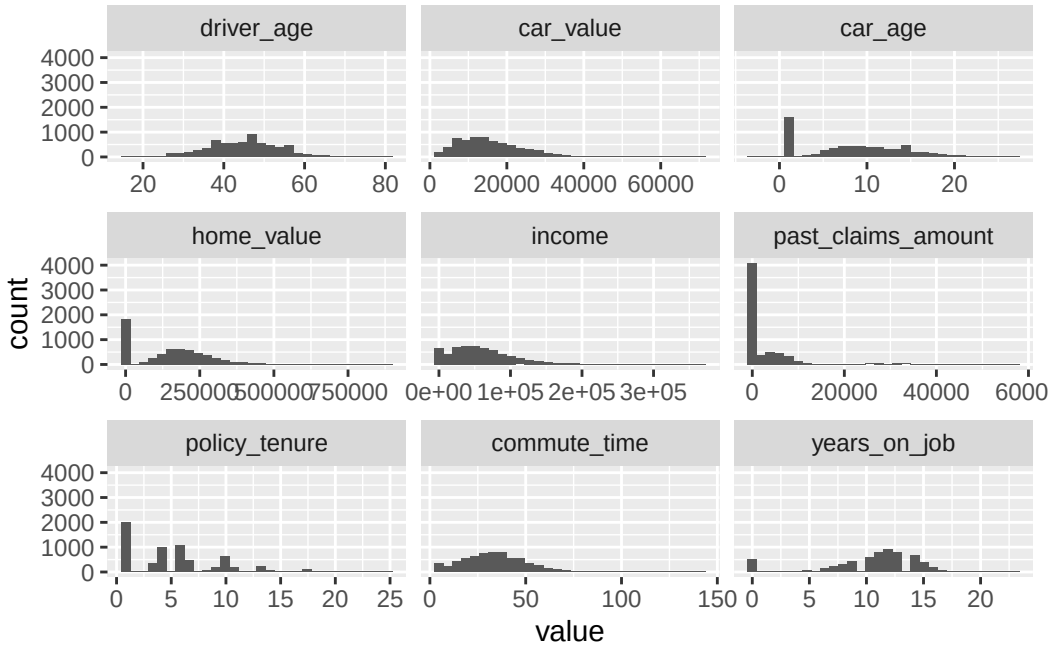
Independent Variables

I have broken up the histograms into two groups. The first group of histograms displayed are the continuous quantitative variables and the second group will contain histograms of the binary and discrete numerical variables. This way, it is much easier to identify patterns in how the data is distributed for each variable.

Group 1: Numerical Continuous Variables

```
# Create clean dataset without the categorical variables with lots of
# categories
clean_df_num <- train_data |> dplyr::select(driver_age, car_value, car_age,
                                            home_value, income, past_claims_amount,
                                            policy_tenure, commute_time,
                                            years_on_job)

# Create a melted dataset to display the histograms of the quantitative

# variables
```

```
df_melted_num <- reshape2::melt(data = clean_df_num)
# Graphing the histograms
ggplot(data = df_melted_num,
       mapping = aes(x = value)) +
  geom_histogram()+
  facet_wrap(~variable, scales = "free_x")
```



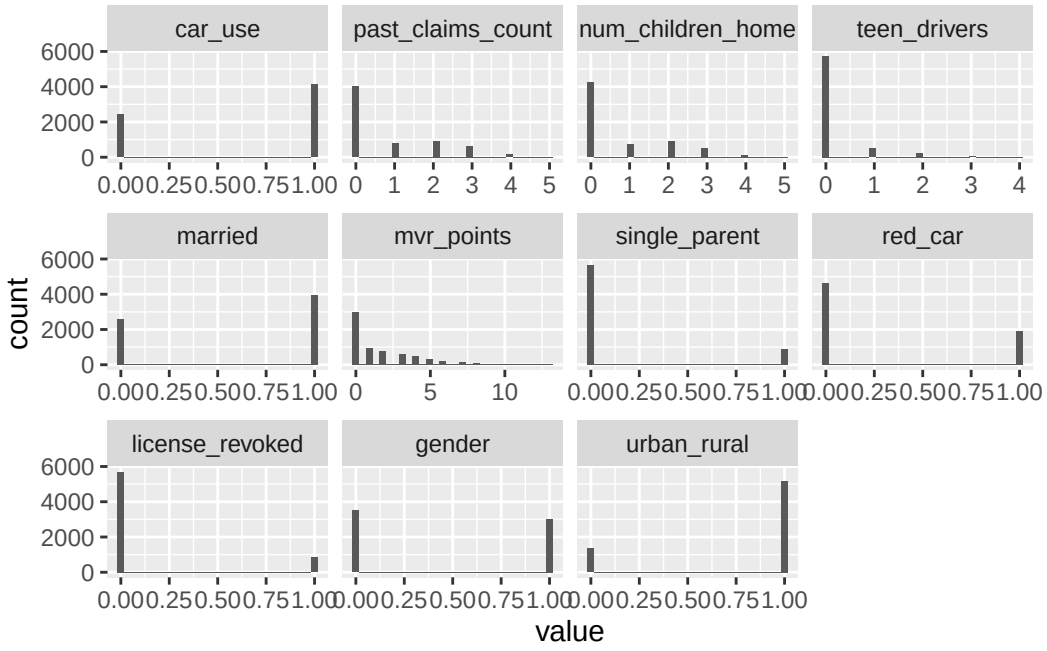
Observations:

- There are lots of right skewed data because the highest frequency for most of the variables is 0.
- In the data preparation section, I will address these skewness problems with log transformations.

Group 2: Binary and Numerical Discrete Variables

```
clean_df_cat <- train_data |> dplyr::select(-crash_cost, -driver_age,
                                            -car_value, -car_age, -home_value,
                                            -income,-past_claims_amount,
                                            -policy_tenure, -commute_time,
                                            -years_on_job, -car_type,
                                            -educ_level, -job_category,
```

```
                                                -crash_flag)

df_melted_cat <- reshape2::melt(data = clean_df_cat)
# Graphing the histograms
ggplot(data = df_melted_cat,
       mapping = aes(x = value)) +
  geom_histogram()+
  facet_wrap(~variable, scales = "free_x")
```



Categorical Variables Bar Charts

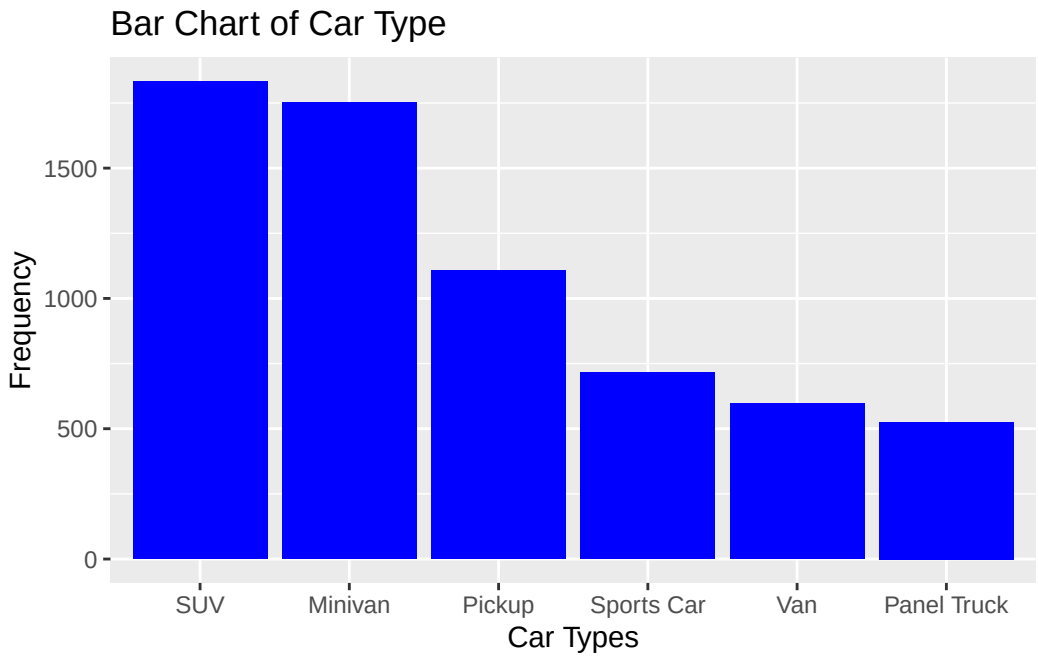
Below, I have recoded the categorical data into factors so that they can be graphed using bar charts. Notice that I deleted the observations that have empty values for `job_category`.

```
# Recode data values in the train dataset first
train_data$car_type <- dplyr::recode(train_data$car_type,
                                     "z_SUV" = "SUV")

train_data$educ_level <- dplyr:: recode(train_data$educ_level,
                                        "<High School" = "High School",
                                        "z_High School" = "High School")
```

```
train_data$job_category <- dplyr::case_when(
  train_data$job_category == "z_Blue Collar"          ~ "Blue Collar",
  is.na(train_data$job_category) | train_data$job_category ==
    ""    ~ "Missing Values",
  TRUE ~ as.character(train_data$job_category)
)


# Create separate data rame for the categorical variables
train_categorical <- train_data |> dplyr::select(car_type, educ_level,
                                                 job_category)
```

Now I will create separate data frame for the frequencies of each categorical for each variable.

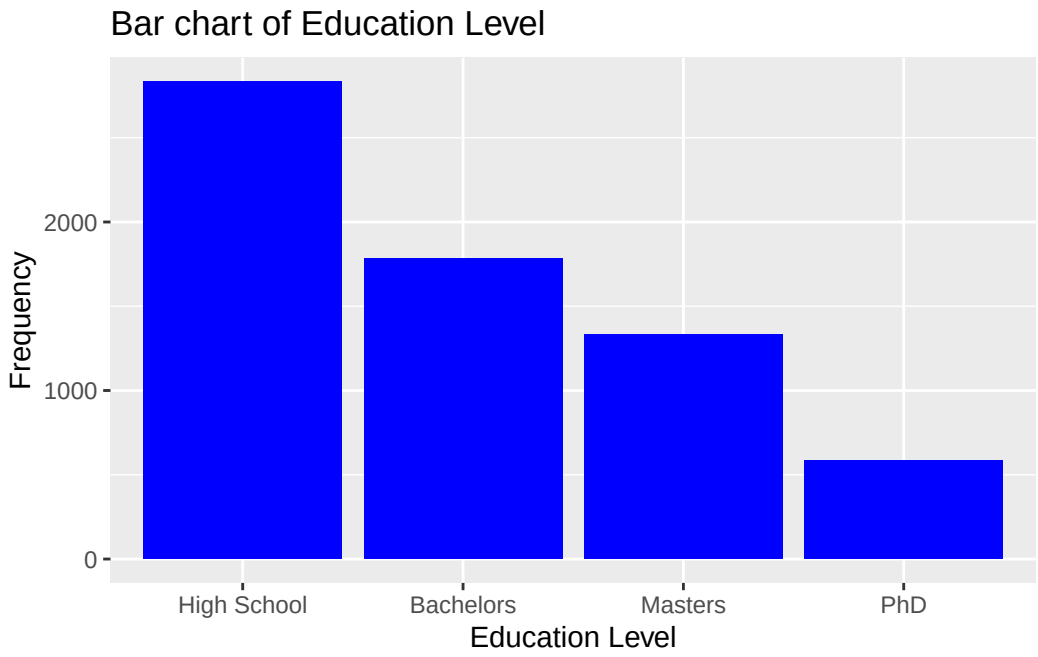
```
# Car Type
car_type_df <- data.frame(Value = names(table(train_categorical$car_type)),
                          Frequency = table(train_categorical$car_type))

# Graph this
ggplot(data = car_type_df,
       mapping = aes(x = reorder(x = Value, X = -Frequency.Freq),
                     y = Frequency.Freq)) +
  geom_bar(stat = "identity", fill = "blue") +
  labs(title = "Bar Chart of Car Type", x = "Car Types", y = "Frequency")
```



- Most of the customers drive SUVs and Minivans
- Very few customers drive vans or trucks

We will do the same for education level.

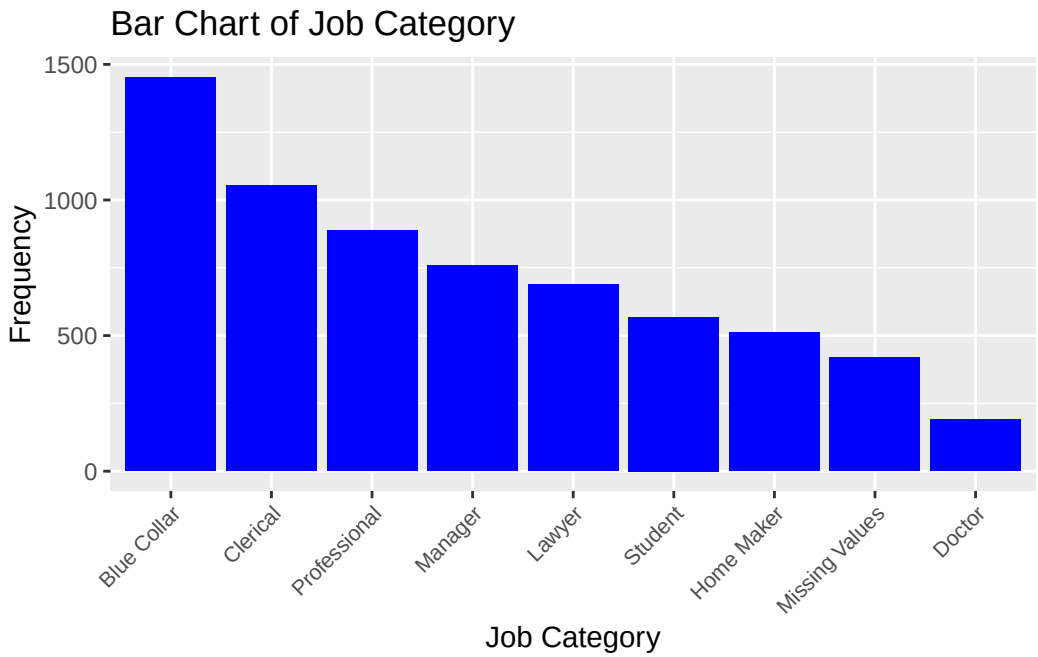
```
educ_level_df <- data.frame(table(train_categorical$educ_level))

# Graph this
ggplot(data = educ_level_df,
       mapping = aes(x = reorder(x = Var1, X = -Freq), y = Freq)) +
  geom_bar(stat = "identity", fill = "blue") +
  labs(title = "Bar chart of Education Level", x = "Education Level",
       y = "Frequency")
```



- The education level category with the most amount of customers is high school
- The majority of customers still have some type of college education.

Now for job category.

```
job_cat_df <- data.frame(table(train_categorical$job_category))
#job_cat_df <- job_cat_df |> dplyr::filter(Var1 != "")


#Graph this
ggplot(data = job_cat_df,
       mapping = aes(x = reorder(x = Var1, X = -Freq), y = Freq)) +
  geom_bar(stat = "identity", fill = "blue") +
  labs(title = "Bar Chart of Job Category", x = "Job Category",
       y = "Frequency") +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1,
      size = 8
    )
  )
```



- The job category with the most amount of customers is Blue Collar
- Note that there are some customers whose job category was not given.

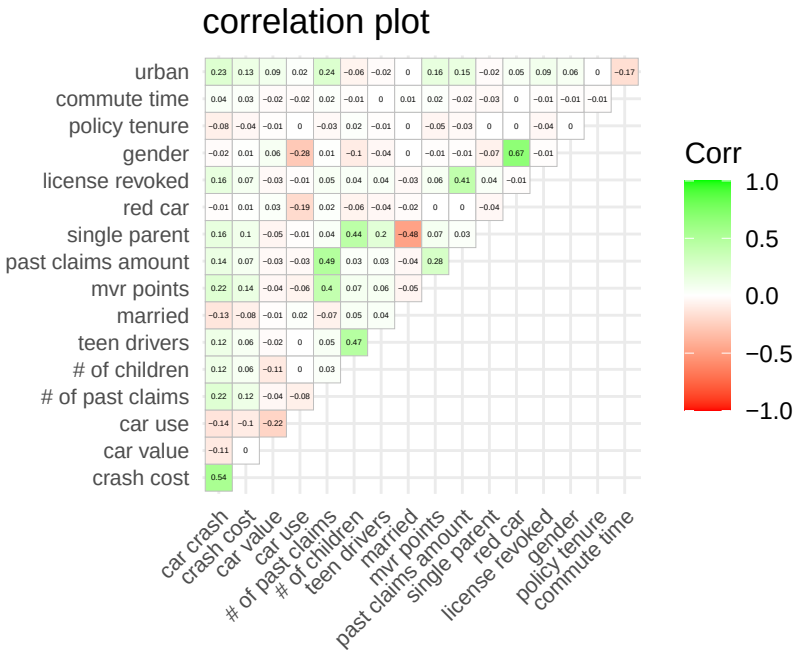
1.4 Correlation Table

Now, I will investigate if there are any explanatory variables that are correlated with either of the response variables `crash_flag` or `crash_cost`. Correlation tables do not work well with categorical data, so I will exclude car type, education level, and job category from the correlation plot.

```
# Create the correlation table with specified variables
corr_tab <- cor(x = train_data[, -c(5, 6, 9, 13)])

colnames(corr_tab) <- c(
  "car crash",
  "crash cost",
  "driver age",
  "car value",
  "car use",
  "# of past claims",
  "# of children",
  "home value",
```

```
  "income",
  "teen drivers",
  "married",
  "mvr points",
  "past claims amount",
  "single parent",
  "red car",
  "license revoked",
  "gender",
  "policy tenure",
  "commute time",
  "urban",
  "years on job"
)

rownames(corr_tab) <- c(
  "car crash",
  "crash cost",
  "driver age",
  "car value",
  "car use",
  "# of past claims",
  "# of children",
  "home value",
  "income",
  "teen drivers",
  "married",
  "mvr points",
  "past claims amount",
  "single parent",
  "red car",
  "license revoked",
  "gender",
  "policy tenure",
  "commute time",
  "urban",
  "years on job"
)

# Use ggcorplot to create the upper correlation plot
corr_plot <- ggcorrplot(corr     = corr_tab,
                        type     = "upper",
```

```
                         method   = "square",
                         title    = "correlation plot",
                         colors   = c("red", "white", "green"),
                         lab      = T,
                         lab_size = 1,
                         insig    = "pch",
                         tl.cex   = 8,
                         digits = 2

                         )
corr_plot
```



Correlation Plot Analysis

- It's clear that both `crash_flag` and `crash_cost` have the same variables that are positively and negatively correlated with them. This makes sense because both of the dependent variables of interest are fairly strongly correlated with a value of 0.54.
  - Most notably, `urban`, `mvr_points`, and `past_claims_count` have the strongest positive correlations with `crash_flag` and `crash_cost`.
  - The most negatively correlated variables with whether or not someone gets into a crash are `married` and `car_use` which makes sense because in theory, married people are typically more responsible and get into less crashes.

- Independent variables correlated among themselves
  - `teen_drivers` and `num_children_home`
    - Positively correlated with a value of 0.47.
    - This makes sense because customers with more children at home are likely to have more teen drivers. Note that including both of these variables may result in multicollinearity.
  - `past_claims_amount` and `past_claims_count`
    - Obvious positive correlation between the number of past claims someone has made and the total amount of money from those claims. Including both variables in the regressions may result in multicollinearity.
  - `mvr_points` and `past_claims_count`
    - Positively correlated with a value of 0.4 which makes sense because people who have more violations are likely to have filed more claims.
  - `gender` and `red_car`
    - The positive correlation value of 0.67 is a very interesting result in theory, we don't think of the color of cars being correlated with gender. This result suggests that more men are linked with driving red cars.
  - `license_revoked` and `past_claims_amount`
    - Positively correlated with a value of 0.41 which makes sense because people who have their license revoked are more likely to have made claims in the past.
  - `single_parent` and `married`
    - negatively correlated which is obvious since single parents are not married and vice versa.

2 Data Preparation

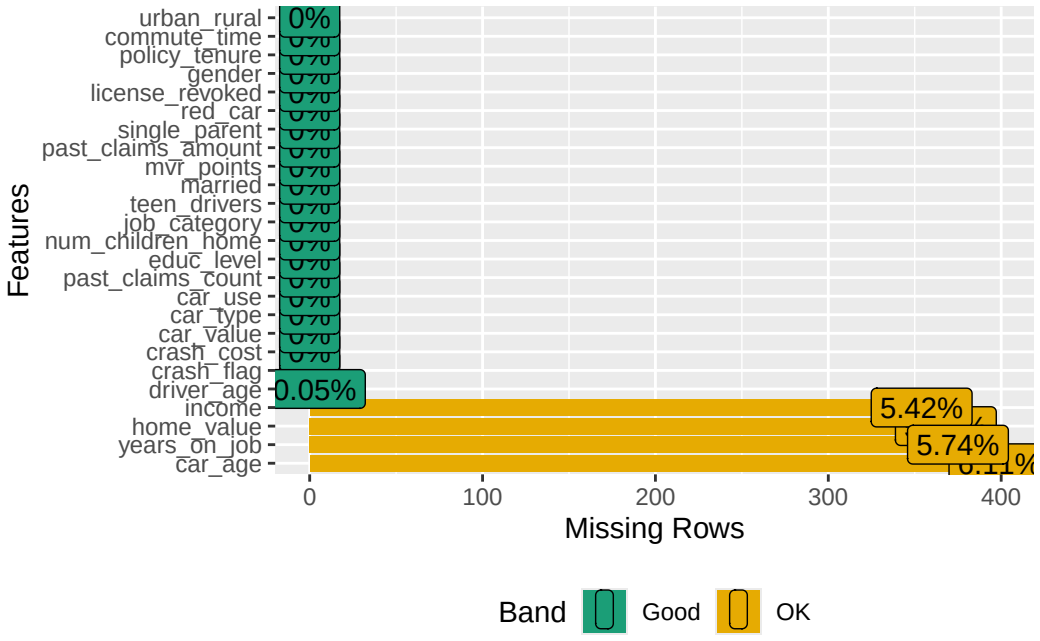
2.1 Missing Values

Identifying the variables that have missing values is an important step for data cleaning. Below, I have outlined the procedure I use on how to address missing values.

- Determine the percentage of values that are missing for each variable that contains missing values.
  - Anything variable that has less than 10% of its observations missing, I will keep those columns.
- Consider the importance of each variable to the problem
- Consider deleting the specific observations with missing values if the data is missing completely at random (MCAR)
  - I will run an `mcar_test` from the naniar package to evaluate this.
- If there is no evidence to suggest that the data is not missing completely at random, then deleting certain missing values or imputing them.

Below is a visual representation of the percentage of missing values that each variable contains.

```
plot_missing(train_data)
```



As we can see, the following variables contain missing values in the training data:

- `driver_age`: 0.05% missing values
  - The age of the driver has a strong theoretical effect on how safe they drive and there are very few missing values. No reason to drop the entire column.
- `income`: 5.42% missing values
  - The income of customers could be very intriguing to study. Stereotypes would suggest that people with higher income are safer drivers, however, more expensive sports cars may be more dangerous.
  - Less than 10% of values are missing and the variable `income` is of interest, so there is no reason to drop this entire column.
- `home_value`: 5.64% of observations have missing values
  - Similar to income, theory points to people with high home values being safer drivers, but yet again, they may own more fancy cars that could be more dangerous. Not many missing values, so no need to remove the entire column.
- `years_on_job`: 5.74% of observations

  - Not many missing values and will be interesting to study since people who have worked at one place for longer may be more responsible, however, they may drive to work more often and put themselves in danger. No reason to drop the entire column.

- `car_age`: 6.11% of observations.
  - Older cars could get into more accidents since they are more likely to have malfunctions. Only about 6% of its values are missing so it would be better to keep this column.

- All of these variables have under 10% of their values missing, and from the graph, we can see that these quantities of missing values are not alarming. Therefore, I have decided to not drop any of these variable columns outright.

  Notice that summing these percentages up gives a value of about 22.96%, meaning that about 23% of the total observations in the training data have at least one missing value. I have confirmed this below:

```
(length(which(is.na(train_data))) / dim(train_data)[1])*100
```

```
[1] 22.96262
```

- Therefore, deleting all rows that have at least one missing value will delete approximately 23% of the observations recorded in the dataset which could significantly bias the results.
- However the missingness map displays that only 1% of all of the total data entries are missing, implying that imputing these missing values should have little to no bias on the results. I will still test below if the missing data is missing completely at random or not.

Missing Completely at Random?

I will run an mcar test to evaluate if any of the missing values are indeed missing at random.

```
naniar::mcar_test(train_data)
```

```
# A tibble: 1 x 4
  statistic    df p.value missing.patterns
      <dbl> <dbl>   <dbl>            <int>
1      402.   390   0.323               18
```

The null hypothesis is that the data is missing completely at random, so the p-value of 0.15>0.05, meaning that we fail to reject the null hypothesis that the data is missing completely at random. Note this does not fully prove that the data is missing completely at random, but

rather there is no strong evidence against this claim. Therefore, implementing median imputation or deleting observations will likely not bias the results from our regressions.

Imputation of Missing Variables

- I have decided to impute all of the missing values using the standard median imputation method. I did consider using the `mice` algorithm that considers the relationships between the variables when deciding how to impute the missing values. However, this method imputes the values with the mean which could cause some bias since we found that lots of the variables are heavily skewed.
- Imputing with the median, although it does not implement the same rigorous algorithm as the mice method, will still be more resistant to the skewness of the data, leading to less biased coefficient estimates for our regressions.

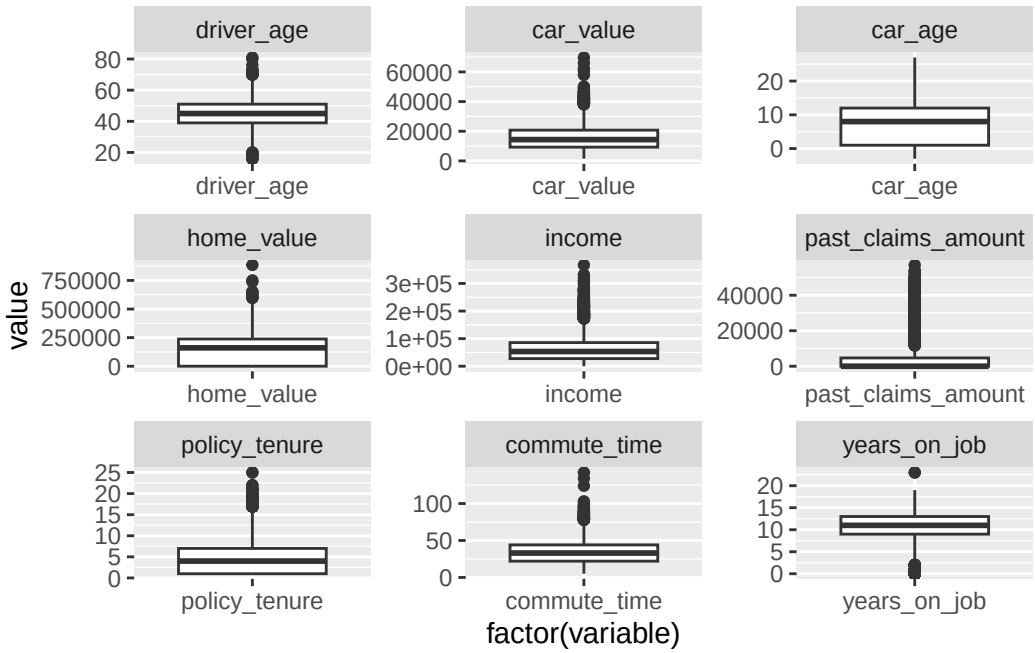
```
# Median imputation
train_data$driver_age[is.na(train_data$driver_age)] <-
median(train_data$driver_age, na.rm = T)

train_data$income[is.na(train_data$income)] <-
median(train_data$income, na.rm = T)

train_data$home_value[is.na(train_data$home_value)] <-
median(train_data$home_value, na.rm = T)

train_data$years_on_job[is.na(train_data$years_on_job)] <-
median(train_data$years_on_job, na.rm = T)

train_data$car_age[is.na(train_data$car_age)] <-
median(train_data$car_age, na.rm = T)
```

After imputing all of the missing values in the dataset, I will now investigate any outlier values and address them in the next section.

2.2 Outlier Values

Now, I must deal with any outlier values that are present in the dataset. To observe which variables have outliers, below I have displayed box plots of the numerical independent variables in the dataset.

```
ggplot(data = df_melted_num,
       mapping = aes(x = factor(variable), y = value)) +
  geom_boxplot() +
  facet_wrap(facets = ~variable, scale = "free")
```



Box Plot Analysis:

- It's clear that almost all of the numerical predictors have some outliers which we will need to address to reduce bias that those values might have in the regressions.
- I do not want to delete any observations with outliers and impute those values with the mean or median because there are a substantial number of outliers in the data.
- Instead, I will transform the variables with large outliers in order to reduce their influence on the data. These variables include:
  - `car_value`, `home_value`, `income`, `past_claims_amount`, `policy_tenure`, and `commute_time`.

2.2.1 Transforming Variables

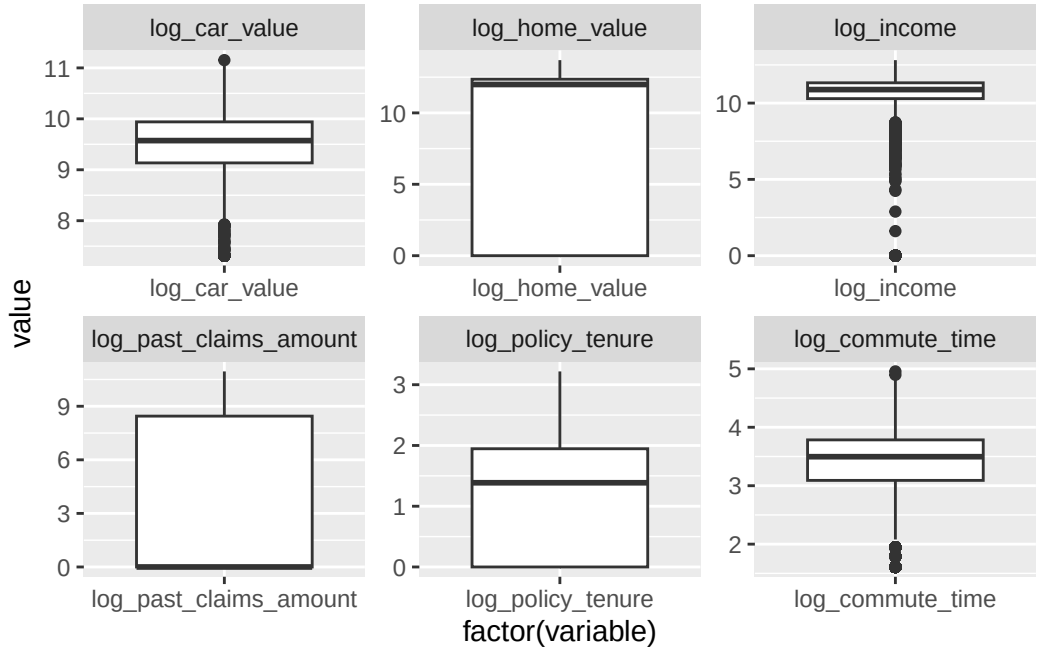
Below I have performed log transformations on the variables that are heavily right-skewed. Note that the original data for some of these variables were zero, meaning that taking the log

would result in a value of negative infinity which would greatly disrupt our results. Therefore, I converted any instances of negative infinity values to zeros.

```
# Car Value
train_data$log_car_value <- log(train_data$car_value)

# home value
train_data$log_home_value <- log(train_data$home_value)
train_data$log_home_value[train_data$log_home_value == -Inf] <- 0

# income
train_data$log_income <- log(train_data$income)
train_data$log_income[train_data$log_income == -Inf] <- 0

# Past claims amount
train_data$log_past_claims_amount <- log(train_data$past_claims_amount)
train_data$log_past_claims_amount[
  train_data$log_past_claims_amount == -Inf] <- 0

# Policy Tenure
train_data$log_policy_tenure <- log(train_data$policy_tenure)

# Commute Time
train_data$log_commute_time <- log(train_data$commute_time)
```

Below are the update box plots of the log transformed variables that I specified previously.

```
clean_df_num_log <- train_data |>
  dplyr::select(log_car_value, log_home_value, log_income,
         log_past_claims_amount, log_policy_tenure, log_commute_time)



# Create a melted dataset to display the histograms of the quantitative
# variables
df_melted_num_log <- reshape2::melt(data = clean_df_num_log)



ggplot(data = df_melted_num_log,
       mapping = aes(x = factor(variable), y = value)) +
  geom_boxplot() +
  facet_wrap(facets = ~variable, scale = "free")
```



From the box plots above, applying log transformations to those six variables appears to have reduced the number of high outliers that was causing many of the variables to be right-skewed. However, notice that there are some new low outliers especially for `log_income` which could be a problem. Also the distributions of some of the variables look odd which may be a problem for our regressions.

With all of the missing values imputed and the outlier values dealt with, we can finally take a look at our new summary statistics.

```
stargazer(train_data, type = "text", covariate.labels = labels,
          median = T, title = "Updated Summary Statistics",
          omit.summary.stat = "N", notes = "N = 6,528")
```

```
Updated Summary Statistics
===============================================================================
Statistic                    Mean       St. Dev.     Min  Median       Max
-------------------------------------------------------------------------------
Crash Indicator             0.264        0.441       0       0          1
Crash Cost                1,466.616    4,545.654   0.000   0.000   107,586.100
Driver Age                 44.853        8.649      16      45         81
```

Vehicle Value	15,641.760	8,381.489	1,500	14,370	69,740
Vehicle Age in years	8.296	5.547	-3	8	27
Vehicle Use	0.631	0.483	0	1	1
Past Claims count	0.799	1.159	0	0	5
Children at Home	0.719	1.106	0	0	5
Home Value	154,658.200	125,197.500	0	160,680	885,282
Annual Income	61,195.250	46,386.770	0	53,276	367,030
Teen Drivers count	0.171	0.509	0	0	4
Marital Status	0.600	0.490	0	1	1
MVR Points	1.700	2.145	0	1	13
Past Claims	4,119.320	8,924.665	0	0	57,037
Single Parent	0.131	0.338	0	0	1
Red Car Indicator	0.292	0.455	0	0	1
License Revoked	0.127	0.333	0	0	1
Driver Gender	0.465	0.499	0	0	1
Policy Tenure in years	5.367	4.150	1	4	25
Commute Time/Distance	33.442	15.966	5	33	142
Urbanicity	0.794	0.405	0	1	1
Years on Job	10.539	3.986	0	11	23
log_car_value	9.491	0.629	7.313	9.573	11.153
log_home_value	8.758	5.495	0.000	11.987	13.694
log_income	10.005	2.991	0.000	10.883	12.813
log_past_claims_amount	3.397	4.320	0.000	0.000	10.951
log_policy_tenure	1.300	0.952	0.000	1.386	3.219
log_commute_time	3.361	0.608	1.609	3.497	4.956

N = 6,528

3 Build Models

Now it's time to construct regression models on the training data that can accurately predict whether or not an insurance customer has been in a crash, and if so, predict how much the crash will cost.

To make predictions on if an individual has been in a crash, I will use logistic regression because the dependent variable is a binary variable (in crash or not in a crash). I will construct three different logistic regression models, interpret each of their results, and then compare their confusion matrices to determine which model is the best one..

For predicting the cost of crashes, I will create one multiple linear regression using stepAIC and I will experiment using Lasso regression to create the second regression model. For these regressions, I will separate the training data so that I only run the regressions on observations of people who have been in car accidents. I will compare the coefficient estimates, their prediction accuracy metrics, and their AIC values to determine which model is the best.

3.1 Logistic Regression

```
train_data$educ_level <- factor(train_data$educ_level, ordered = F)
train_data$car_type <- factor(train_data$car_type, ordered = F)
```

3.1.1 Logistic Model 1

In my first logistic regression model I will manually select variables that I believe to most accurately predict whether a customer is involved in a crash. My estimating equation for model 1 is seen below:

$$\begin{aligned} P(crash.flag_i = 1) = \beta_0 + \beta_1 \cdot driver.age_i + \beta_2 \cdot car.age_i + \\ \beta_3 \cdot past.claims.count_i + \beta_4 \cdot income_i + \\ \beta_5 \cdot teen.drivers_i + \beta_6 \cdot licsense.revoked_i + \\ \beta_7 \cdot policy.tenure_i + u_i \end{aligned}$$

Variable Reasons

In my explanatory data analysis, I identified `driver_age` and `income` as two main variables that I wanted to explore.

I also included `past_claims_amount` and `license_revoked` because in theory, people who have filed more claims in the past and who have had their license revoked may be more likely to get into accidents.

I also included `teen_drivers` because younger drivers are assumed to be less responsible, and customers with more teen drivers therefore may have a higher likelihood of getting into a crash. Lastly, I included `policy_tenure` to have a variable that may have the opposite effect crash likelihood. as the previous variables I mentioned. People who have been customers for longer may be more responsible drivers and less likely to be involved in a crash.

I have run the first regression below, and I will compare its regression output to the next two models.

```
log_reg1 <- glm(
  data = train_data,
  formula = crash_flag ~ driver_age + car_age + past_claims_count + income +
    teen_drivers + license_revoked + policy_tenure,
  family = binomial(link = "logit")
)
```

3.1.2 Logistic Model 2

The next model I will create will include every variable in the data set. My goal here is to generate a model and then use backward selection to find the best model with the lowest AIC. The estimating equation for the full model is seen below:

P

$$\begin{aligned} P(crash.flag_i = 1) = \beta_0 + \beta_1 \cdot driver.age_i + \beta_2 \cdot car.value_i + \beta_3 \cdot car.age_i + \\ \delta_2 \cdot car.type2_i + \cdots + \delta_6 \cdot car.type6_i + \\ \beta_4 \cdot car.use_i + \beta_5 \cdot past.claims.count_i + \\ \gamma_2 \cdot educ.level2_i + \cdots + \gamma_5 \cdot educ.level5_i + \beta_6 \cdot num.children.home_i + \\ \beta_7 \cdot home.value_i + \beta_8 \cdot income_i + \ \sigma_2 \cdot job.category2_i + \cdots + \\ \sigma_9 \cdot job.category9_i + \beta_9 \cdot teen.drivers_i + \beta_{10} \cdot married_i + \\ \beta_{11} \cdot mvr.points_i + \beta_{12} \cdot past.claims.amount_i + \beta_{13} \cdot single.parent_i + \\ \beta_{14} \cdot red.car_i + \beta_{15} \cdot license.revoked_i + \beta_{16} \cdot gender_i + \beta_{17} \cdot policy.tenure_i + \\ \beta_{18} \cdot commute.time_i + \beta_{19} \cdot urban.rural_i + \beta_{20} \cdot years.on.job_i + \\ \beta_{21} \cdot log.car.value_i + \beta_{22} \cdot log.home.value_i \\ \beta_{23} \cdot log.income_i + \beta_{24} \cdot \log .past.claims.amount_i + \\ \beta_{25} \cdot log.policy.tenure_i + \beta_{26} \cdot log.commute.time_i + u_i \end{aligned}$$

- $\delta_2 ... \delta_6$ represent the coefficients for each car type dummy variable.
- $\gamma_2 ... \gamma_5$ represent the coefficients for each education level dummy variable.
- $\sigma_2 ... \sigma_9$ represent the coefficients for each job category dummy variable.

```
log_reg2 <- glm(data = train_data,
                formula = crash_flag ~ . -crash_cost,
                family = binomial(link = "logit"))
```

3.1.3 Logistic Model 3

Now, I will use backward selection from my full model to develop my third logistic regression model. Below is the estimating equation for this model:

$$\begin{aligned} P(crash.flag_i = 1) = {} & \delta_2 \cdot car.type2_i + \cdots + \delta_6 \cdot car.type6_i + \\ & \beta_1 \cdot car.use_i + \gamma_2 \cdot educ.level2_i + \cdots + \gamma_5 \cdot educ.level5_i + \\ & \beta_2 \cdot income_i + \sigma_2 \cdot job.category2_i + \cdots + \sigma_9 \cdot job.category9_i + \\ & \beta_3 \cdot teen.drivers_i + \beta_4 \cdot married_i + \\ & \beta_5 \cdot mvr.points_i + \beta_6 \cdot past.claims.amount_i + \beta_7 \cdot single.parent_i + \\ & \beta_8 \cdot license.revoked_i + \beta_9 \cdot gender_i + \\ & \beta_{10} \cdot urban.rural_i + \beta_{11} \cdot log.car.value_i + \beta_{12} \cdot log.home.value_i \\ & \beta_{13} \cdot log.income_i + \beta_{14} \cdot \log .past.claims.amount_i + \\ & \beta_{15} \cdot log.policy.tenure_i + \beta_{16} \cdot log.commute.time_i + u_i \end{aligned}$$

```
back_reg <- stepAIC(object = log_reg2,
                    direction = c("backward"))
```

```
log_reg3 <- glm(data = train_data,
                formula = crash_flag ~ car_type + car_use + educ_level
                + income + job_category + teen_drivers + married
                + mvr_points + past_claims_amount + single_parent +
                  license_revoked + urban_rural + log_car_value +
                  log_home_value + log_income + log_past_claims_amount
                + log_policy_tenure + log_commute_time,
                family = binomial(link = "logit"))
```

3.1.4 Logistic Regression Coefficient Comparison

Below, I have displayed the coefficient estimates for the three logistic regression models.

```r
# Create covariate labels for the regression table


stargazer(log_reg1, log_reg2, log_reg3, type = "text", digits = 2,
          title = "Logistic Regression Results",
          column.labels = c("Reg 1", "Reg 2", "Reg 3"),
          dep.var.labels = c("Crash Flag"),
          covariate.labels = c("Driver Age", "Car Value", "Car Age",
                               "Panel Truck", "Pickup", "Sports Car", "Van",
                               "SUV", "Car Use", "Past Claims count",
                               "Bachelors", "Masters", "PhD",
                               "Number of Children home", "Home Value",
                               "income", "Clerical Worker", "Doctor",
                               "Home Maker", "Lawyer", "Manager",
                               "Missing Values", "Professional", "Student",
                               "Teen Drivers", "Married", "MVR Points",
                               "Past Claims Amount", "Single Parent",
                               "Red Car", "License Revoked", "Gender",
                               "Policy Tenure", "Commute Time",
                               "Urban or Rural", "Years on Job",
                               "log Car Value", "log Home Value",
                               "log Income", "log Past Claims Amount",
                               "log Policy Tenure", "log Commute Time"))
```

```
Logistic Regression Results
=============================================================
                              Dependent variable:
                          -------------------------------
                                      Crash Flag
                            Reg 1       Reg 2       Reg 3
                             (1)         (2)         (3)
-------------------------------------------------------------
Driver Age                -0.02***     -0.004
                          (0.003)     (0.005)

Car Value                              0.0000
                                      (0.0000)

Car Age                   -0.02***     0.005
                           (0.01)      (0.01)
```

Panel Truck		0.31*	0.48***
		(0.18)	(0.16)
Pickup		0.59***	0.57***
		(0.11)	(0.11)
Sports Car		1.05***	0.91***
		(0.15)	(0.12)
Van		0.56***	0.62***
		(0.14)	(0.14)
SUV		0.88***	0.74***
		(0.13)	(0.10)
Car Use		-0.74***	-0.73***
		(0.10)	(0.10)
Past Claims count	0.38***	0.04	
	(0.02)	(0.05)	
Bachelors		-0.46***	-0.44***
		(0.10)	(0.09)
Masters		-0.29	-0.26
		(0.18)	(0.16)
PhD		-0.23	-0.19
		(0.22)	(0.20)
Number of Children home		0.02	
		(0.04)	
Home Value		0.0000	
		(0.0000)	
income	-0.0000***	-0.0000**	-0.0000**
	(0.0000)	(0.0000)	(0.0000)
Clerical Worker		-0.0005	-0.003
		(0.12)	(0.12)
Doctor		-1.22***	-1.23***

		(0.34)	(0.34)
Home Maker		-0.37**	-0.40**
		(0.19)	(0.18)
Lawyer		-0.34	-0.35*
		(0.21)	(0.21)
Manager		-0.92***	-0.94***
		(0.16)	(0.16)
Missing Values		-0.42**	-0.43**
		(0.21)	(0.21)
Professional		-0.23*	-0.24*
		(0.13)	(0.13)
Student		-0.50***	-0.47***
		(0.17)	(0.16)
Teen Drivers	0.40***	0.46***	0.47***
	(0.05)	(0.07)	(0.06)
Married		-0.48***	-0.44***
		(0.10)	(0.09)
MVR Points		0.10***	0.10***
		(0.02)	(0.02)
Past Claims Amount		-0.0000***	-0.0000***
		(0.0000)	(0.0000)
Single Parent		0.39***	0.46***
		(0.12)	(0.11)
Red Car		-0.02	
		(0.10)	
License Revoked	0.89***	1.01***	1.02***
	(0.08)	(0.10)	(0.10)
Gender		0.19	
		(0.13)	

Policy Tenure	-0.05***	-0.02	
	(0.01)	(0.02)	
Commute Time		0.01	
		(0.01)	
Urban or Rural		2.33***	2.32***
		(0.13)	(0.13)
Years on Job		0.01	
		(0.01)	
log Car Value		-0.46***	-0.34***
		(0.14)	(0.06)
log Home Value		-0.03**	-0.03***
		(0.01)	(0.01)
log Income		-0.07***	-0.06***
		(0.02)	(0.02)
log Past Claims Amount		0.07***	0.09***
		(0.02)	(0.01)
log Policy Tenure		-0.18**	-0.24***
		(0.09)	(0.03)
log Commute Time		0.23	0.36***
		(0.15)	(0.06)
Constant	-0.06	1.35	0.14
	(0.16)	(1.22)	(0.65)
Observations	6,528	6,528	6,528
Log Likelihood	-3,427.87	-2,885.99	-2,890.02
Akaike Inf. Crit.	6,871.74	5,857.98	5,844.04
Note:		*p<0.1; **p<0.05; ***p<0.01	

Before interpreting the coefficients, it is important to discuss the log likelihood and AIC metrics that we see at the bottom of the.

The log-likelihood is used to calculate the AIC for logistic regressions, and lower values of AIC indicate a better logistic regression model. The log-likelihood itself is also a measure of a model's performance and high values for log-likelihood are more desirable. According to the regression results, model 3 has the lowest AIC value of 5,844 and model 2 has the highest log-likelihood value of -2885.99.

It is difficult to decide which of these models is the best based on these measures, so I will decide after constructing the confusion matrices for each model.

Coefficient Interpretations:

I will go ahead and interpret some of the coefficient estimates for model 2 since this is one of the two best models for prediction. Since there are so many variables in these regressions, I will only interpret one continuous variable, one binary variable, and I will also make sure that I interpret both a positive and negative coefficient estimate.

- **Coefficient Sign**
  - `license_revoked`(+): The coefficient sign is positive which is expected since people who have had their license revoked are more likely to get into an accident.
  - `mvr_points`(+): The coefficient sign is positive which makes sense because people with more tickets or traffic violations are more likely to get into a crash.
  - `married(-)`: The coefficient sign is negative, suggesting that people who are married are less likely to get into an accident. This makes sense because we traditionally think of married people as more responsible, and thus safer drivers.
- **Economic magnitude**
  - `license_revoked`: $\beta_{15} = 1.01$: This means that when a customer has had their license revoked before, the log-odds of getting in a crash increase by 1.01, holding all else constant. In other words, when we take $e^{1.01} = 2.75$, the odds of a customer who has had their licence revoked getting into a crash increase by a factor of about 2.75. This is a major result, because this means that people who have had their license revoked are more than double as likely to get into an accident than people who have not.
  - `mvr_points`: $\beta_{11} = 0.10$: For every additional ticket a customer receives, the log-odds of getting into a crash increase by about 0.10, holding all other factors constant. Alternatively, taking $e^{0.10} = 1.105$, for every additional ticket a customer receives, the odds of a crash occurring increases by a factor of about 1.105. This is not a very significant result, however if a customer has many ticket violations, we can predict that they will be much more likely to get into an accident.

- `married`: $\beta_4 = -0.48$. The log-odds of a customer getting into an accident decreases by -0.48 when the customer is married versus if a customer is single. Alternatively, taking $e^{-0.48} = 0.62$ means that the odds of getting into an accident decreases by a factor of 0.62 or 38% for married customers. This is a major result because it can help us determine whether a customer is likely to get in an accident based on their marital status..

- **Statistical Significance**

  - The coefficient estimates for `license_revoked`, `mvr_points`, and `married` are all statistically significant below the significance level of $\alpha = 0.01$ level. This means that the coefficient estimates most likely did not come about randomly, and thus, we can be very confident in the coefficient estimates.

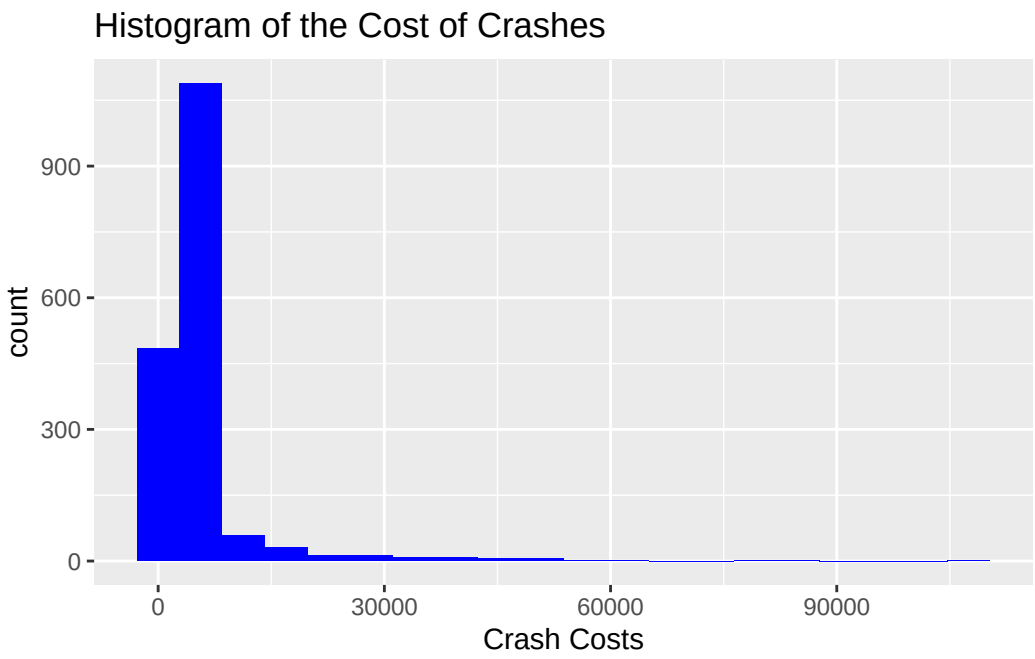
3.2 Linear Regressions

Now, we will shift our attention to predicting the accident costs for customers who got into a car crash. The cost of accidents is a continuous numerical variable, therefore, we will incorporate a multivariate linear regression model to predict the cost of accidents. Also, since we are only focusing on people involved in car accidents, I have created a new data frame that only includes observations who were involved in an accident.

```
df_crash <- train_data |> dplyr::filter(crash_flag == 1)
```

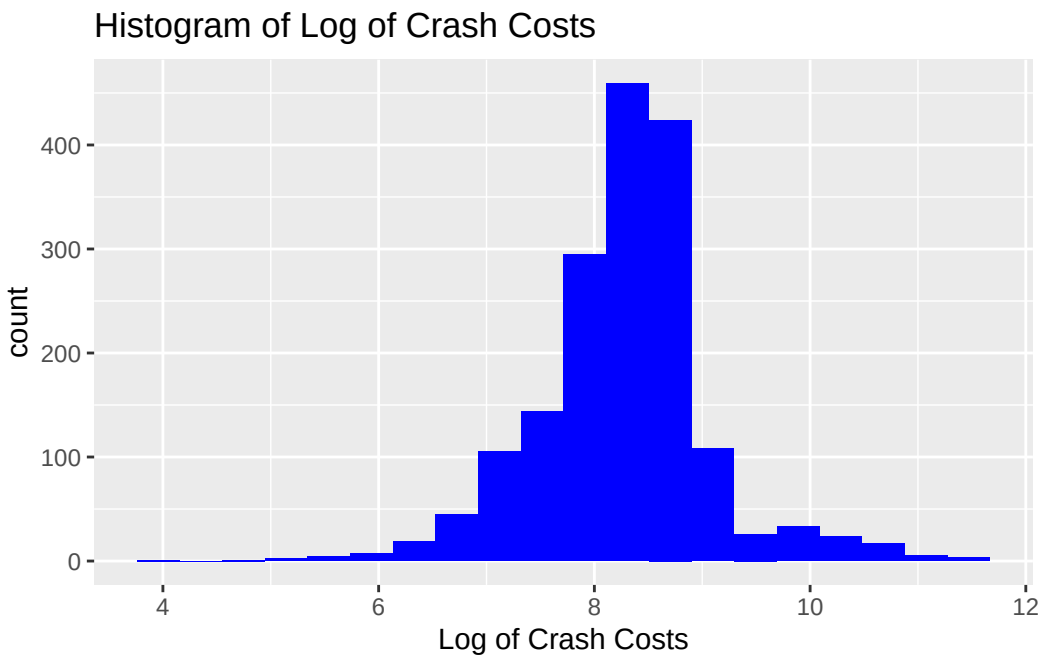
Below is a histogram of the cost of crashes which is very skewed to the right.

```
ggplot(data = df_crash,
       mapping = aes(x = crash_cost)) +
  geom_histogram(fill = "blue", bins = 20) +
  labs(title = "Histogram of the Cost of Crashes",
       x = "Crash Costs")
```



Use a log transformation on the dependent variable `crash_cost` to address its outliers and control for the skewness.

```
df_crash$log_crash_cost <- log(df_crash$crash_cost)

ggplot(data = df_crash,
       mapping = aes(x = log_crash_cost)) +
  geom_histogram(fill = "blue", bins = 20) +
  labs(title = "Histogram of Log of Crash Costs",
       x = "Log of Crash Costs")
```



The histogram illustrates a much more symmetric distribution of the costs of the crashes. Now let's run some multiple linear regressions using the log transformation of the crash costs to make predictions on how much a crash will cost.

3.2.1 OLS Linear Regression Model 1

The first model I will build is an OLS multiple linear regression model that will contain all of the variables to predict the cost accidents. I will then use backward selection to construct the second linear regression model. My estimating equation for the full regression model is the following:

$$
\begin{aligned}
log.crash.cost_i = \beta_0 + \beta_1 \cdot driver.age_i + \beta_2 \cdot car.value_i + \beta_3 \cdot car.age_i + \\
\delta_2 \cdot car.type2_i + \cdots + \delta_6 \cdot car.type6_i + \\
\beta_4 \cdot car.use_i + \beta_5 \cdot past.claims.count_i + \\
\gamma_2 \cdot educ.level2_i + \cdots + \gamma_5 \cdot educ.level5_i + \beta_6 \cdot num.children.home_i + \\
\beta_7 \cdot home.value + \beta_8 \cdot income + \ \sigma_2 \cdot job.category2_i + \cdots + \\
\sigma_9 \cdot job.category9_i + \beta_9 \cdot teen.drivers_i + \beta_{10} \cdot married_i + \\
\beta_{11} \cdot mvr.points_i + \beta_{12} \cdot past.claims.amount + \beta_{13} \cdot single.parent_i + \\
\beta_{14} \cdot red.car_i + \beta_{15} \cdot license.revoked_i + \beta_{16} \cdot gender_i + \beta_{17} \cdot policy.tenure + \\
\beta_{18} \cdot commute.time_i + \beta_{19} \cdot urban.rural_i + \beta_{20} \cdot years.on.job_i + \\
\beta_{21} \cdot log.car.value_i + \beta_{22} \cdot log.home.value_i \\
\beta_{23} \cdot log.income_i + \beta_{24} \cdot \log .past.claims.amount_i + \\
\beta_{25} \cdot log.policy.tenure_i + \beta_{26} \cdot log.commute.time_i + u_i
\end{aligned}
$$

```
lin_full <- lm(data = df_crash, formula = log_crash_cost ~ . -crash_flag
               -crash_cost)
```

3.2.2 OLS Linear Regression Model 2

The second model I will build is an OLS multiple linear regression model. I will use backward selection using stepAIC to determine the best linear model.

```
back_linear <- MASS::stepAIC(object = lin_full,
                             direction = c("backward"))
```

After performing backward selection, the model with the lowest AIC contained only the following five variables:

- `married`, `mvr_points`, `gender`, `log_car_value`, and `log_home_value`

Based on the results from using **stepAIC**, my estimating equation for model 2 is the following:

$$
\begin{aligned}
log.crash.cost_i = \beta_0 + \beta_1 \cdot married_i + \beta_2 \cdot mvr.points_i + \beta_3 \cdot gender_i \\
+ \beta_4 \cdot log.car.value_i + \beta_5 \cdot log.home.value + u_i
\end{aligned}
$$

```
lin_reg1 <- lm(data = df_crash,
                      formula = log_crash_cost ~ married + mvr_points + gender
               + log_car_value + log_home_value)
```

3.2.3 Linear Regression Coefficient Comparisons

Now, we will compare the coefficient results for the multiple linear regressions we built using the Lasso and the setpAIC feature selection methods. The coefficient estimate results are seen in the table below.

```
linear_labels = c(
  ""
)
stargazer(lin_full, lin_reg1, type = 'text',
          title = "Regression Results for Linear Models",
          digits = 2, column.labels = c("Full Model", "setpAIC Model"),
          dep.var.labels = c("Log Cost of Crash"),
          covariate.labels = c("Driver Age", "Car Value", "Car Age",
                               "Panel Truck", "Pickup", "Sports Car",
                               "Van", "SUV", "Car Use",
                               "Past Claims count", "Bachelors",
                               "Masters", "PhD", "Number of Children home",
                               "Home Value", "income", "Clerical Worker",
                               "Doctor", "Home Maker", "Lawyer", "Manager",
                               "Missing Values", "Professional", "Student",
                               "Teen Drivers", "Married", "MVR Points",
                               "Past Claims Amount", "Single Parent",
                               "Red Car", "License Revoked", "Gender",
                               "Policy Tenure", "Commute Time",
                               "Urban or Rural", "Years on Job",
                               "log Car Value", "log Home Value",
                               "log Income", "log Past Claims Amount",
                               "log Policy Tenure", "log Commute Time"))
```

```
Regression Results for Linear Models
=================================================================
                                  Dependent variable:
                              ---------------------------------------
                                       Log Cost of Crash
                                Full Model          setpAIC Model
                                   (1)                  (2)
-----------------------------------------------------------------
Driver Age                         0.002
                                  (0.002)
```

Car Value	-0.0000
	(0.0000)
Car Age	-0.005
	(0.01)
Panel Truck	0.12
	(0.11)
Pickup	0.03
	(0.07)
Sports Car	0.03
	(0.09)
Van	-0.04
	(0.09)
SUV	0.07
	(0.08)
Car Use	0.01
	(0.06)
Past Claims count	-0.04
	(0.03)
Bachelors	-0.02
	(0.06)
Masters	0.13
	(0.11)
PhD	0.18
	(0.14)
Number of Children home	0.01
	(0.02)
Home Value	-0.0000
	(0.0000)
income	-0.0000

	(0.0000)	
Clerical Worker	-0.005	
	(0.07)	
Doctor	0.06	
	(0.24)	
Home Maker	-0.003	
	(0.11)	
Lawyer	-0.02	
	(0.13)	
Manager	-0.01	
	(0.10)	
Missing Values	-0.04	
	(0.13)	
Professional	0.06	
	(0.08)	
Student	0.11	
	(0.09)	
Teen Drivers	-0.05	
	(0.04)	
Married	-0.10*	-0.10**
	(0.06)	(0.05)
MVR Points	0.02*	0.02**
	(0.01)	(0.01)
Past Claims Amount	0.0000	
	(0.0000)	
Single Parent	0.04	
	(0.07)	
Red Car	0.02	
	(0.06)	

License Revoked	-0.08	
	(0.06)	
Gender	0.09	0.06
	(0.08)	(0.04)
Policy Tenure	0.01	
	(0.01)	
Commute Time	-0.0003	
	(0.004)	
Urban or Rural	0.05	
	(0.09)	
Years on Job	-0.01	
	(0.01)	
log Car Value	0.26***	0.15***
	(0.08)	(0.03)
log Home Value	0.01*	0.01
	(0.01)	(0.004)
log Income	0.01	
	(0.01)	
log Past Claims Amount	0.005	
	(0.01)	
log Policy Tenure	-0.04	
	(0.05)	
log Commute Time	-0.002	
	(0.09)	
Constant	5.73***	6.78***
	(0.70)	(0.27)
Observations	1,722	1,722
R2	0.04	0.02

```
Adjusted R2                      0.01                    0.02
Residual Std. Error        0.79 (df = 1679)       0.78 (df = 1716)
F Statistic             1.52** (df = 42; 1679) 8.57*** (df = 5; 1716)
=========================================================================
Note:                                        *p<0.1; **p<0.05; ***p<0.01
```

Coefficient Interpretations

I will use the second model for coefficient interpretations because it has the lowest AIC, so I will most likely use this model for predictions on the evaluation dataset.

- **Coefficient Signs**
  - `married` has a negative coefficient, indicating that crashes that involve unmarried people may be more expensive than when married people get in accidents.
  - `mvr_points` has a positive coefficient which makes sense because people who have lots of traffic tickets are likely to pay more when they get into an accident.
  - `gender` has a positive coefficient which makes sense because men are probably more dangerous drivers and thus have to pay more when they get into an accident.
  - `log_car_value` has a positive coefficient which is expected since more expensive cars will result in a higher payout when crashes occur.
  - `home_value` has a positive coefficient which is interesting because we normally think of wealthier people as more responsible drivers, however, if a person who has a high home value does get into an accident, that could play a role in how expensive the crash becomes.
- **Economic Magnitude**
  - `married` : $\beta_1 = -0.10$: Since the dependent variable is the log transformation of the cost of the crashes, then the interpretations will be in percentages. On average, married people pay about 10% less than unmarried people in the event of an accident, holding all else constant. This is a pretty significant effect and it can allow us to predict that crashes involving married customers will probably not be as expensive.
  - `mvr_points`: $\beta_2 = 0.02$: On average, for every additional violation from an insurance customer, the cost of the crash increases by about 2%, holding everything else constant. This doesn't seem like a significant effect, however, if we know that someone has a large number of tickets, this can be helpful to predict the cost of the crash.

  - `gender`: $\beta_3 = 0.06$: In the event of a crash, if the customer is a male, the cost of the crash will be about 6% more than if the customer is a female, holding everything else constant. This makes sense because men are usually seen as more reckless and thus face more severe penalties when involved in accidents.
  - `log_car_value`: $\beta_4 = 0.15$: In the event of a crash, for every 1% increase in the car value, the cost of a crash will increase by about 15%, holding all else constant. This is a major effect and it tells us that crashes involving expensive cars will result in higher payouts.
  - `log_home_value`: $\beta_5 = .01$: In the event a crash, for every additional 1% increase in the home value of the customers, the cost of a crash increases by about 1%. This is not a very significant result in terms of the magnitude.

- **Statistical Significance**
  - The coefficient estimates for the variables `married`, `mvr_points`, and `log_car_value` are all statistically significant under the level $\alpha = 0.05$.
  - This indicates that these estimates are most likely not to have randomly occurred, and thus, we are confident in the accuracy of these coefficient results.
  - `gender` and `log_home_value` are not statistically significant, meaning that we are not confident in the accuracy of these coefficient estimates and may have occurred by randomness.

4 Select Models

4.1 Logistic Regression Model Selection

I will construct confusion matrices for all three models and compare them to determine which logistic regression is the best model. Before constructing the confusion matrices, I will first make predictions on the training data using the three logistic regressions.

```
# Predict probabilities for each regression model
train_data$pred_log1 <- predict(log_reg1, type = "response")
train_data$pred_log2 <- predict(log_reg2, type = "response")
train_data$pred_log3 <- predict(log_reg3, type = "response")

# Create Threshold at 0.5
threshold = 0.5


# indicated classes for the observations using the threshold (1 for crash
# and 0 for no crash)
train_data$logit1_class <- ifelse(train_data$pred_log1 >= threshold, 1, 0)
train_data$logit2_class <- ifelse(train_data$pred_log2 >= threshold, 1, 0)
train_data$logit3_class <- ifelse(train_data$pred_log3 >= threshold, 1, 0)
```

Now that we have our predictions for the three logistic regression models, we can construct the confusion matrices to evaluate which one has the best prediction accuracy.

Diagnostic Testing

```
library(caret)
conf1<- confusionMatrix(as.factor(train_data$logit1_class),
                as.factor(train_data$crash_flag), positive = "1")

conf2 <- confusionMatrix(as.factor(train_data$logit2_class),
                as.factor(train_data$crash_flag), positive = "1")

conf3 <- confusionMatrix(as.factor(train_data$logit3_class),
                as.factor(train_data$crash_flag), positive = "1")
```

```
conf_table <- data.frame(
  Model = c("Model 1", "Model 2", "Model 3"),
  Accuracy = c(conf1$overall["Accuracy"],
               conf2$overall["Accuracy"],
```

```
                conf3$overall["Accuracy"]),
  Classification_Error = c(1 - conf1$overall["Accuracy"],
                           1 - conf2$overall["Accuracy"],
                           1 - conf3$overall["Accuracy"]),
  Sensitivity = c(conf1$byClass["Sensitivity"],
                  conf2$byClass["Sensitivity"],
                  conf3$byClass["Sensitivity"]),
  Specificity = c(conf1$byClass["Specificity"],
                  conf2$byClass["Specificity"],
                  conf3$byClass["Specificity"]),
  Precision = c(conf1$byClass["Pos Pred Value"],
                conf2$byClass["Pos Pred Value"],
                conf3$byClass["Pos Pred Value"])
)

conf_table |> kable(caption = "Comparison of the Confusion Matrix Metrics",
                    digits = 3)
```

Table 2: Comparison of the Confusion Matrix Metrics

Model	Accuracy	Classification_Error	Sensitivity	Specificity	Precision
Model 1	0.746	0.254	0.165	0.954	0.561
Model 2	0.794	0.206	0.435	0.923	0.670
Model 3	0.792	0.208	0.429	0.922	0.664

Confusion Matrix Analysis

- Accuracy
  - Model 2 slightly has the highest accuracy at 79.4% correct classifications of whether someone got in an accident or not.
- Classification Error
  - Classification error is just one minus the accuracy, so therefore, Model 2 also has the lowest classification error.
- Sensitivity (True positive Rate)
  - Model 2 has the highest sensitivity at 43.5%. This means for the people who actually got into accidents, our model only correctly classified 43.5% as getting into an accident.

- Specificity (True negative Rate)
  - Model 1 has the highest specificity at 95.4%. Model 2 has a specificity of about 92.3%. This means that out of the customers who did not get into a crash, model 1 correctly identified 95.4% of them as not getting into an accident and model 2 correctly classified 92.3% of people who were not in accidents..
- Precision (Positive Predicted Value)
  - Model 2 has the highest precision value at about 67%. This means that out of the observations our model classified as getting into an accident, about 67% of those classifications were correct.

Based on the above metrics, I have decided to choose **Model 2** to make predictions on the evaluation data. Model 2 has the highest log-likelihood, the highest accuracy, the highest sensitivity, and the highest precision. Therefore, it will likely be the most suitable for prediction on the evaluation dataset.

4.2 Linear Regression Model Selection

I will compare the mean squared error, F-statistic values, and the R-squared values for the two linear regression models I built. I will then graph their residual plots to compare which of the models better follow the Gauss-Markov assumptions. After considering all of the diagnostic tests, I will make a decision on which linear regression model to use to predict the cost of the crashes in the evaluation data.

```
diag_table <- data.frame(
  model = c("Full Model", "Model 2"),
  MSE = c(mean(lin_full$residuals**2), mean(lin_reg1$residuals**2)),
  R_squared = c((summary(lin_full))$adj.r.squared,
                (summary(lin_reg1))$adj.r.squared),
  f_stat = c((summary(lin_full))$fstatistic[1],
             (summary(lin_reg1))$fstatistic[1])
)
diag_table |> kable(caption = "Comparison of Diagonostic Testing",
                    digits = 2)
```

Table 3: Comparison of Diagonostic Testing

model	MSE	R_squared	f_stat
Full Model	0.61	0.01	1.52
Model 2	0.61	0.02	8.57

MSE

- The full model has a slightly lower mean squared error than the second model, however this is not a very big difference, and in fact, they show up as identical when rounding to the second decimal place in the table above.

R-Squared

- Both adjusted R-squared values are very low with the full model being at 0.01 whereas the model 2 is at 0.02. This means that the models only explain about 1-2% of the variability that is observed in the cost of the crashes.

F-statistic

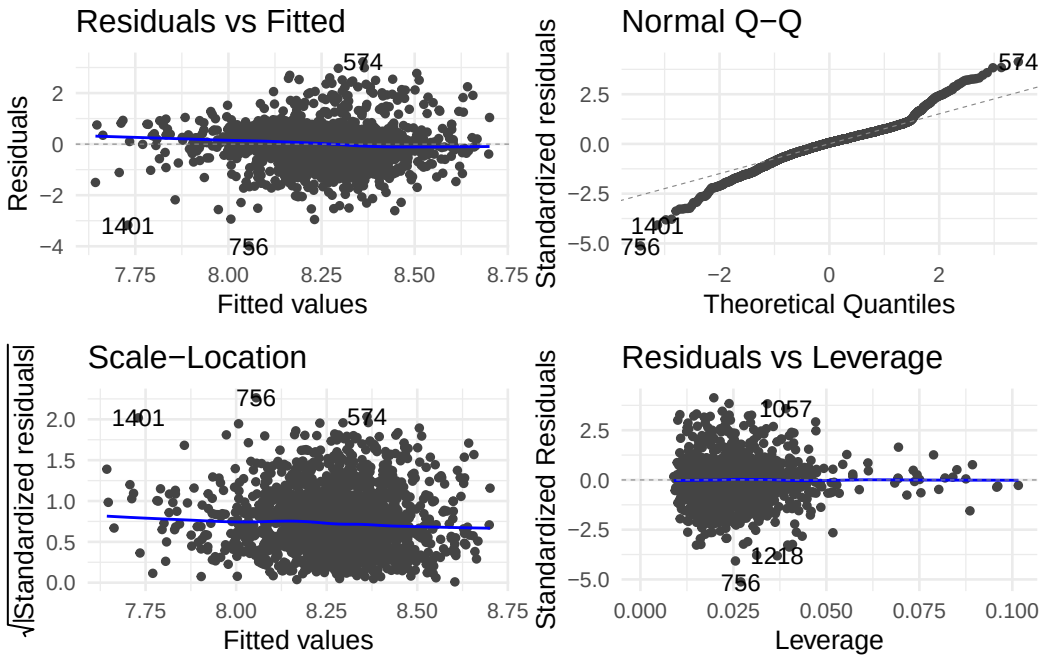
- Both F-statistic values are relatively low, however, the F-statistic value for the second model is about 8.57 which is higher than the F-stat value of 1.52 for the full model. The F-statistic value for model 2 is statistically significant under the $\alpha = 0.01$ level, implying that the model as a whole is statistically significant and that we are confident there is a relationship between the predictors and the cost of a crash.

Residual Analysis Plots

Below are the residual plots on the training data for both the full model and the updated model using backward selection.

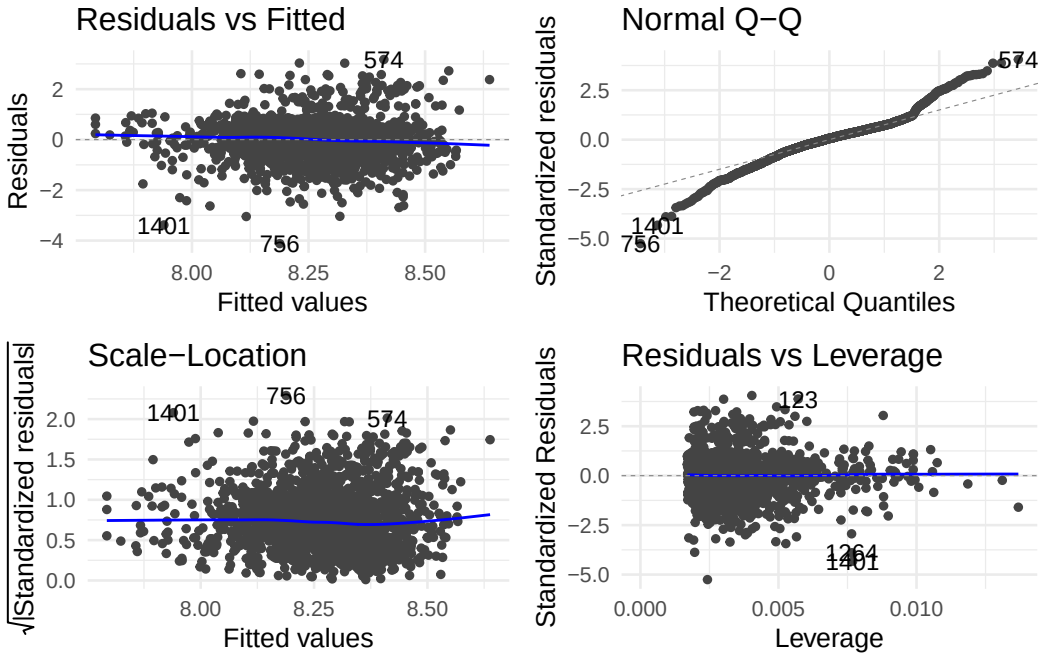
Full Linear Model Residual Plots

```
autoplot(lin_full,label.size =3,size =1)+theme_minimal(base_size =10)
```



Linear Model 2 Residuals Plots

```
autoplot(lin_reg1, label.size = 3, size = 1)+theme_minimal(base_size = 10)
```



Residual Plot Analysis

All of the residual plots for both the full and the shortened models are all very similar to each other. Below, I have provided a summary of what each of the four residual plots means in regards to the models obeying the Gauss-Markov assumptions.

- **Residual vs Fitted**
  - The residual vs fitted plots both show the residuals to be randomly scattered around zero with no systematic curvature.
  - The blue regression lines are linear around zero which indicates that the assumption of linearity holds.
- **Normal Q-Q**
  - The residuals are relatively normally distributed for theoretical quantile values around zero.
  - However, the residuals do shift away from the normal distribution near the tails, which is likely a result of the skewness of some of the data.
- **Scale-Location**
  - The scale-location plot for `lin_reg1` appears to exhibit less heteroscedasticity than for the full model.
  - It appears that the homoscedasticity assumption holds for the second model since the scale-location plot shows constant variance among the residuals.
- **Residuals vs Leverage**
  - There are no major indications of leverage points. There are some data points labeled that are far away from zero, but the blue regression line is still linear, indicating they have little effect on either of the models

Testing the VIF for both models

Our last diagnostic test to determine which model is the best will be seeing if either model has multicollinearity by looking at the variance inflation factor for both models. Below are two tables of the VIF values for each model.

```
vif1 <- vif(lin_full)
#Model 2
vif2 <- vif(lin_reg1)
#Model 3
```

```
#vif_table |> kable(caption = "Comparison of the Variance Inflation Factor")

vif1[,1] |> kable (caption = "VIF For Full Linear Model", col.names = "VIF")
```

Table 4: VIF For Full Linear Model

	VIF
driver_age	1.544652
car_value	9.990124
car_age	2.034179
car_type	8.185516
car_use	2.334061
past_claims_count	3.126323
educ_level	9.214506
num_children_home	2.360495
home_value	7.625297
income	4.401080
job_category	41.662806
teen_drivers	1.473977
married	2.350962
mvr_points	1.191089
past_claims_amount	2.808207
single_parent	2.195859
red_car	1.846706
license_revoked	1.636857
gender	3.917239
policy_tenure	7.264878
commute_time	8.029088
urban_rural	1.064013
years_on_job	2.676284
log_car_value	7.405589
log_home_value	7.000381
log_income	4.863951
log_past_claims_amount	4.838392
log_policy_tenure	7.257288
log_commute_time	7.964766

```
vif2 |> kable(caption = "VIF for Model 2", col.names = "VIF")
```

Table 5: VIF for Model 2

	VIF
married	1.471499
mvr_points	1.004295
gender	1.006821
log_car_value	1.017753
log_home_value	1.481523

VIF Analysis

- Overall, the full model has higher VIF values that indicate there is likely multicollinearity which could result in some bias for the coefficient estimates. There is one very high VIF value for `job_category`
- The VIF values for the second linear model are very low with all values below 2. This suggests that the second Gauss-Markov assumption of no perfect multicollinearity is not violated, thus, we can trust our coefficient estimates.

Linear Model Selection for Predictions on Evaluation Data

Based on the combination of the diagnostic tests with MSE, adjusted R-squared, the f statistic, the residual plots, and the VIF test for multicollinearity, I have decided to use **Linear Model 2** to make predictions on the cost of car accidents in the evaluation data.

4.3 Logistic Model Evaluation Data Predictions

To make predictions for both the logistic and linear regressions on the evaluation dataset, I first had to clean the evaluation data so it would be compatible with the models I trained on the training data. This process involved the same steps for renaming variables, imputing missing values, and log transformations that I performed on the training data. For this reason, I have hidden the code below to make the report more concise and presentable.

```
eval_data$pred_log <- predict(log_reg2, newdata = eval_data,
                              type = "response")
```

```
eval_data$logit_class <- ifelse(eval_data$pred_log >= threshold, 1, 0)
```

```r
confpred <- confusionMatrix(as.factor(eval_data$logit_class),
                            as.factor(eval_data$crash_flag),
                positive = "1")

confpred_table <- data.frame(
  model = c("Model 2"),
  Accuracy = c(confpred$overall["Accuracy"]),
  Classification_Error = c(1 - confpred$overall["Accuracy"]),
  Sensitivity = c(confpred$byClass["Sensitivity"]),
  Specificity = c(confpred$byClass["Specificity"]),
  Precision = c(confpred$byClass["Pos Pred Value"])
)


confpred$table
```

```
          Reference
Prediction    0    1
         0 1105  246
         1   97  185
```

```r
confpred_table |> kable(caption = "Confusion matrix Prediction Metrics",
                        digits = 3, row.names = F)
```

Table 6: Confusion matrix Prediction Metrics

model	Accuracy	Classification_Error	Sensitivity	Specificity	Precision
Model 2	0.79	0.21	0.429	0.919	0.656

The table above displays the classification accuracy metrics for logistic Model 2 for the evaluation data.

- Accuracy:
  - Our best logistic model trained on the training data has a prediction accuracy of 79% on the evaluation data set. This means that it correctly classified 79% of the unseen customers as either getting into a crash or not getting into a crash.
- Classification Error**:**
  - The model incorrectly classified 21% of the observations in the evaluation data.

- Sensitivity (True Positive Rate)
  - The true positive rate is about 43%, meaning that out of the customers in the evaluation data who actually got into an accident, our logistic model correctly classified 43% of those individuals.
- Specificity (True Negative Rate)
  - The true negative rate is about 92%, meaning that out of the customers in the evaluation data who did not get into an accident, our logistic model correctly classified 92% of those individuals as not getting into a crash.
- Precision (Positive Predicted Vale)
  - The positive predicted value is 65.6%, meaning that out of the customers our logistic model classified as getting into an accident, 65.6% of those insurance customers actually did get into a crash.

4.4 Linear Regression Evaluation Data Model Predictions

I created a separate data frame for just predicting the cost of the crashes for people who were in a crash. I also created a log transformation of the crash cost variable so that the evaluation data can be suitable for our model to make predictions.

```
df_eval_crash <- eval_data |> dplyr::filter(crash_flag == 1)

df_eval_crash$log_crash_cost <- log(df_eval_crash$crash_cost)
```

Now I will make predictions on the log cost of accidents for customers in the evaluation data and compare the results to the actual log values of the car accident costs.

```
df_eval_crash$pred_cost <- predict(lin_reg1, newdata = df_eval_crash)
```

I have added the predicted log crash costs to the data frame. Below is a comparison of the summary statistics for the actual log car crash values and the predicted log crash values for Model.

```
log_cost_comp <- data.frame(
  Model = c("Predictions", "Actual"),
  Mean = c(mean(df_eval_crash$pred_cost), mean(df_eval_crash$log_crash_cost)),
  Median = c(median(df_eval_crash$pred_cost),
             median(df_eval_crash$log_crash_cost)),
```

```
  Min = c(min(df_eval_crash$pred_cost), min(df_eval_crash$log_crash_cost)),
  Max = c(max(df_eval_crash$pred_cost), max(df_eval_crash$log_crash_cost))
)

log_cost_comp |> kable(caption = "Summary Statistics Comparison for Log
                       predicted Crash Costs versus actual log crash costs",
                       digits = 2)
```

Table 7: Summary Statistics Comparison for Log predicted Crash Costs versus actual log crash costs

Model	Mean	Median	Min	Max
Predictions	8.27	8.28	7.88	8.67
Actual	8.31	8.32	3.41	11.21

Table Analysis

- The mean and median log crash cost values are very similar, indicating that our model's predictions of the log crash costs of the accidents are very accurate
- Using the log transformation of the crash costs in our model and for the actual crash costs helped reduce the influence of outliers which allowed for better prediction accuracy.
- The min and max values are much more spread apart for the actual crashes, indicating more potential outliers and likely skewness. However, taking the log helped mitigate these effects which are why the mean and median values are so close to each other for the predictions and actual values.

I have also exponentiated the log predicted costs to get a more intuitive idea about what our model predicts to be the normal costs of these accidents. Below, is a table comparing the summary statistics between the exponentiation of the log predicted costs and the actual crash cost values.

```
cost_comp <- data.frame(
  Model = c("Predictions", "Actual"),
  Mean = c(mean(exp(df_eval_crash$pred_cost)), mean(df_eval_crash$crash_cost)),
  Median = c(median(exp(df_eval_crash$pred_cost)),
             median(df_eval_crash$crash_cost)),
  Min = c(min(exp(df_eval_crash$pred_cost)), min(df_eval_crash$crash_cost)),
  Max = c(max(exp(df_eval_crash$pred_cost)), max(df_eval_crash$crash_cost))
)
```

```
cost_comp |> kable(caption = "Summary Statistics Comparison between Actual
                  and Predicted crash costs for Evaluation Data", digits = 2)
```

Table 8: Summary Statistics Comparison between Actual and Predicted crash costs for Evaluation Data

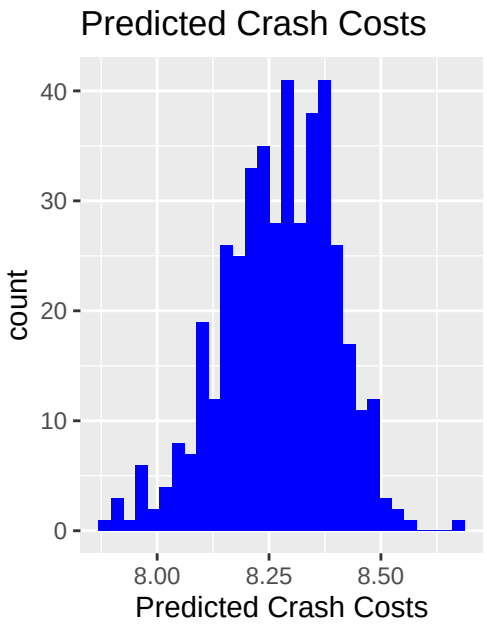
Model	Mean	Median	Min	Max
Predictions	3939.48	3952.93	2632.75	5807.00
Actual	6270.82	4093.00	30.28	73783.47

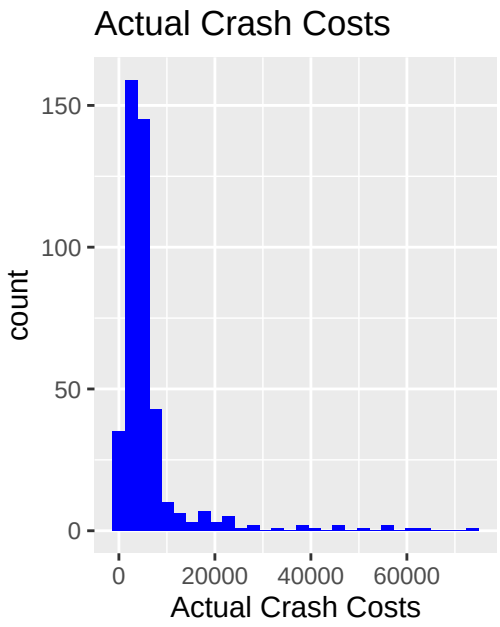
Prediction Analysis

- The median predicted crash cost value from our model is about $3,952.93 which is very close to the actual median crash cost value of $4,093.
- The mean value for the actual crash costs is much higher because of the high outliers in the actual data. This can be seen with the wide dispersion between the min and max values.
- The distribution for the exponentiated predicted values is much more symmetric as seen with the similar median and mean values.

Below is a comparison of the distribution of the predicted crash costs and the actual crash costs. We can see that the predictions of the car crash costs are much more normally distributed than the actual costs of the accidents.

```
hist1 <- ggplot(data = df_eval_crash,
       mapping = aes(x = pred_cost)) +
  geom_histogram(fill = "blue") +
  labs(title = "Predicted Crash Costs",
       x = "Predicted Crash Costs")

hist2 <- ggplot(data = df_eval_crash,
       mapping = aes(x = crash_cost)) +
  geom_histogram(fill = "blue") +
  labs(title = "Actual Crash Costs",
       x = "Actual Crash Costs")

hist1 + hist2
```


Predicted Crash Costs
count
40
30
20
10
0
8.00
8.25
8.50
Predicted Crash Costs


Actual Crash Costs
count
150
100
50
0
0
20000
40000
60000
Actual Crash Costs

Conclusion

- In this project, we looked at an auto insurance dataset that contained important information about the demographics of customers and whether they had been in an accident and how much the accident cost if they were in an accident.
- I trained three logistic regression models to predict the likelihood that someone was involved in a crash.
- After comparing the logistic models using confusion matrix metrics, I determined that **logistic model 2** was the best model for prediction on the evaluation data set. This model contained every variable in the dataset, including the log transformations that I created in the data cleaning section.
- The following are notable variables that were statistically significantly associated with getting in more accidents:
  - People who work in an urban area
  - Single parents
  - Unmarried people
  - People who drive sports cars
  - Customers who have lots of teen drivers
  - People with more traffic tickets and traffic violations
- This logistic model had a **79% accuracy rate**, meaning that it correctly classified 79% of the observations in the evaluation data as getting into an accident or not.
- I also trained two multivariate linear regression models to predict the cost of crashes for customers who were involved in accidents.
- After running various diagnostic tests and residual analysis, I decided that **linear model 2** was the best model for prediction on the evaluation data.
- The following are the most notable variables that were significantly associated with higher crash payouts:
  - Unmarried Customers
  - People with lots of traffic tickets or traffic violations
  - High value of the car involved
- The mean and median value of the predicted log crash cost values are 8.27 and 8.28 respectively. After exponentiating the predicted log crash costs, the mean and median predicted crash costs are $3,939 and $3,952 respectively.